

Откуда они взялись и когда появились

В те времена оперативная память (на магнитных сердечниках) была невероятно дорогой и крошечной — счет шел на килобайты. Если программа или ее данные не помещались в ОЗУ, программисту приходилось вручную писать код, который сохранял куски данных на магнитный барабан (аналог жесткого диска), а потом загружал их обратно. Это называлось **оверлеями**. Ошибка в логике ломала всю систему, а перенос программы на компьютер с чуть большим объемом памяти требовал полной перегрузки и переписывания кода. [1]

Чтобы решить эту проблему, инженеры решили переложить управление памятью с плеч человека на плечи железа и операционной системы.

Хронология появления:

1. 1959–1962 гг. — Страничная организация (Paging)

Впервые виртуальная память появилась в Великобритании в Манчестерском университете при создании суперкомпьютера **Atlas** (запущен в 1962 году). Ее изобретатель Том Килберн придумал делить память на фиксированные «блоки» (страницы). Компьютер имел всего 16 КБ быстрой памяти, но благодаря страницам программисты видели целых 96 КБ. [1]

2. 1965 г. — Сегментная организация (Segmentation)

Программисты быстро поняли, что резать память «вслепую» на одинаковые куски неудобно для защиты данных и работы компиляторов. В 1965 году в операционной системе **Multics** (совместный проект MIT, GE и Bell Labs) и на мейнфреймах Burroughs B5000 была реализована сегментация. Память стали выделять под логические структуры (отдельно для кода, отдельно для стека).

3. Конец 1960-х — Сегментно-страничная организация

Поняв, что у сегментов ужасная фрагментация, а у страниц — проблемы с логикой, создатели мейнфреймов (включая IBM System/360 Model 67 в 1966-1967 гг.) объединили подходы. Они начали делить логические

сегменты на физические страницы. Эта схема стала золотым стандартом и перекочевала в микропроцессоры, которыми мы пользуемся сегодня (начиная с Intel 80386 в 1985 году).

Шаг 1: Базовая концепция виртуальной памяти (Зачем она нужна)

Прежде чем погружаться в таблицы и адреса, поймите базовый принцип: «Программа видит сплошной виртуальный адрес, а железо переводит его в кусочки физического».

- *Что усвоить:* Понятия «виртуальный адрес», «физический адрес» и роль MMU (Memory Management Unit).

Шаг 2: Страничная организация (Paging)

Это фундамент. Поймите, как устроен самый простой и математически точный дележ памяти. [\[1\]](#)

- *Что усвоить:* Что такое страница (в виртуальной памяти) и фрейм (в физической ОЗУ). Как устроена **Таблица страниц (Page Table)**.
- *Главная задача:* Научиться «на бумаге» переводить виртуальный адрес в физический, зная размер страницы (например, 4 КБ) и смещение. [\[1, 2\]](#)

Шаг 3: Сегментная организация (Segmentation)

Взгляните на память глазами программиста, а не математика.

- *Что усвоить:* Почему программисту удобнее видеть отдельно сегмент кода (запрещенный для изменения) и сегмент данных.
- *Главная задача:* Понять разницу между фиксированным размером (страница) и динамическим (сегмент), а также узнать, что такое *внешняя фрагментация* (когда свободная память есть, но она разбита на мелкие бесполезные щели). [\[1, 2\]](#)

Шаг 4: Сегментно-страничная организация (Комбинированная)

Финальный этап, объединяющий два мира.

- *Что усвоить:* Как виртуальный адрес сначала ищет нужный сегмент, а внутри этого сегмента выбирает нужную страницу.

Шаг 5: Реальность (Многоуровневые таблицы и TLB)

Когда классика изучена, разберитесь, как это работает в вашем ПК прямо сейчас.

- *Что усвоить:* В современных 64-битных процессорах таблицы страниц стали гигантскими, поэтому они стали **многоуровневыми** (дерево таблиц). Чтобы процессор не тормозил при каждом поиске адреса, инженеры придумали сверхбыстрый аппаратный кэш адресов — **TLB (Translation Lookaside Buffer)**.

Давайте разберем фундамент. Чтобы понять, зачем нужна виртуальная память, проще всего представить, как компьютеры работали **до неё**, и с какими кошмарами сталкивались программисты.

Эпоха «без виртуальной памяти» (Прямая адресация)

Раньше программы работали с физической оперативной памятью напрямую. Если в компьютере стояло 4 МБ памяти, у каждого байта был свой жесткий адрес от 0 до 4 000 000.

Это приводило к трем огромным проблемам:

1. **Проблема жестких адресов:** Если Программа А при компиляции была настроена записывать свои данные строго по адресу 1 000 000, вы не могли запустить вторую копию этой программы. Они бы просто перезаписывали данные друг друга и намертво вешали компьютер.
2. **Проблема безопасности (Полная анархия):** Любая программа могла прочитать или изменить память операционной системы или другой запущенной программы.

Ошибившись в коде (например, в цикле), программист мог случайно затереть кусок системных данных.

3. **Физический потолок:** Если игре требовалось 6 МБ памяти, а у вас было только 4 МБ, игра просто отказывалась запускаться.

Суть концепции: Великий Обман

Виртуальная память — это **слой абстракции** между программами и реальными чипами ОЗУ. Операционная система и процессор объединяются, чтобы провернуть гениальный обман.

Они говорят каждой запущенной программе: *«Ты на свете одна. В твоём личном распоряжении есть огромный, сплошной, непрерывный массив памяти (например, целых 4 ГБ). Начинай адресацию с нуля и ни в чём себе не отказывай!»*

В итоге рождаются два разных типа адресов:

- **Виртуальный адрес** — это адрес, который видит и использует программа в своём коде.
- **Физический адрес** — это реальное место (ячейка) внутри чипа оперативной памяти на материнской плате.

Три главные задачи, которые решает виртуальная память

1. Изоляция процессов (Безопасность)

Каждая программа живет в своём «виртуальном коконе». Если Программа №1 и Программа №2 обращаются по виртуальному адресу 1000, они не пересекаются. Для Программы №1 этот адрес превратится в физическую ячейку 54320, а для Программы №2 — в ячейку 98760.

Программы физически **не могут заглянуть в память друг друга**, потому что у них просто нет инструментов, чтобы сформировать чужой физический адрес. Если программа сойдёт с ума, она сломает только себя, но не всю ОС.

2. Динамическое перемещение

Программисту больше не нужно думать, в какой именно свободный слот памяти загрузится его код. ОС может раскидать куски одной программы по разным углам физической оперативки, где нашлось место. Для самой программы память останется красивой, сплошной и непрерывной.

3. Выход за рамки ОЗУ (Файл подкачки)

Появляется та самая возможность, о которой мы говорили в начале: если физическая память кончилась, неактивные куски виртуальной памяти незаметно для программы сбрасываются на жесткий диск или SSD. Программа продолжает думать, что она находится в ОЗУ.

Главный исполнитель: Диспетчер памяти (MMU)

Как этот обман работает на практике? Внутри процессора есть специальный аппаратный чип — **MMU (Memory Management Unit)** или диспетчер управления памятью.

Когда программа говорит: «*Хочу прочитать данные из виртуального адреса X*», MMU мгновенно перехватывает этот запрос, заглядывает в свои таблицы перевода и превращает его в запрос: «*Процессор, читай из физического адреса Y*».

Этот перевод происходит аппаратно и настолько быстро (за наносекунды), что сама программа даже не замечает, как её виртуальные координаты превратились в реальные.

Выглядит этот процесс так:

1. Процессор подает на вход MMU виртуальный адрес
2. Если MMU выключено или если виртуальный адрес попал в нетранслируемую область, то физический адрес просто приравнивается к виртуальному
3. Если MMU включено и виртуальный адрес попал в транслируемую область, производится трансляция адреса, то есть замена номера виртуальной страницы на номер соответствующей ей физической страницы (смещение внутри страницы одинаковое):
 - Если запись с нужным номером виртуальной страницы есть в TLB, то номер физической страницы берется из нее же
 - Если нужной записи в TLB нет, то приходится искать ее в таблицах страниц, которые операционная система размещает в нетранслируемой области ОЗУ (чтобы не было промаха TLB при обработке предыдущего промаха). Поиск может быть реализован как аппаратно, так и программно — через обработчик исключения, называемого страничной ошибкой (page fault). Найденная запись добавляется в TLB, после чего команда, вызвавшая промах TLB, выполняется снова.

Суть страничной организации

Представьте, что виртуальная память программы — это длинный бумажный свиток с кодом. Если оперативка забита, нам сложно найти в ней один сплошной кусок под весь этот свиток. Что делает страничная организация? Она берет ножницы и аккуратно нарезает этот свиток на абсолютно одинаковые кусочки.

- **Виртуальная страница (Page):** Кусочек фиксированного размера, на который нарезается виртуальная память процесса.

- **Физический фрейм (Page Frame):** Точно такой же по размеру кусочек, но уже внутри реальной физической оперативки (ОЗУ). В физической памяти фреймы — это просто пронумерованные «слоты».

Важное правило: Размер страницы ВСЕГДА равен размеру фрейма. Чаще всего в современных процессорах это **4 Килобайта (4096 байт)**.

Как работает «магия» размещения

Благодаря тому, что все кусочки одинаковые, операционная система может раскидать виртуальные страницы одной программы по физическим фреймам ОЗУ **в абсолютно случайном порядке**, как ей удобно.

Пример:

У нас есть программа из 3 страниц: Страница 0, Страница 1 и Страница 2.

В ОЗУ они могут лечь так:

- Страница 0 попадет во Фрейм 5
- Страница 1 попадет во Фрейм 2
- Страница 2 попадет во Фрейм 9

Для самой программы память осталась непрерывной (страницы идут по порядку: 0, 1, 2). А в реальности они разбросаны по всей оперативке.

Главный секрет: Таблица страниц (Page Table)

Чтобы процессор не запутался, где чья страница лежит, операционная система для каждой программы создает **Таблицу страниц**. Это простая шпаргалка для процессора (а точнее, для блока MMU).

Выглядит она примерно так:

Номер виртуальной страницы	Номер физического фрейма в ОЗУ	Статус (В ОЗУ или на Диске?)
0	Фрейм 5	В ОЗУ
1	Фрейм 2	В ОЗУ
2	Фрейм 9	В ОЗУ
3	Сброшена в файл подкачки	На Диске (SSD)

Как адрес превращается из виртуального в физический (на пальцах)

Когда программа хочет прочитать данные, она не просит «страницу 1». Она просит конкретный байт. Поэтому любой виртуальный адрес внутри процессора логически делится на две части:

1. **Номер страницы** (какой кусок свитка нам нужен).
2. **Смещение (Offset)** (какой конкретно по счету байт внутри этого куска нам нужен).

Как работает MMU (математика процесса):

Представим, что программа запрашивает данные по виртуальному адресу. MMU берет этот адрес и смотрит:

1. Ага, **номер страницы** равен 1.
2. MMU заглядывает в *Таблицу страниц* этой программы и видит: Страница 1 сейчас находится во **Фрейме 2**.
3. MMU берет физический адрес Начала Фрейма 2 и прибавляет к нему то же самое **смещение**, которое просила программа.
4. Физический адрес готов! Процессор мгновенно считывает нужный байт из ОЗУ.

Почему это быстро? Смещение менять не нужно! Поскольку размер страницы и фрейма одинаковый (например, 4096 байт), 100-й байт на виртуальной странице останется 100-м байтом и внутри физического фрейма. MMU меняет только «приставку» (номер страницы на номер фрейма).

Что происходит, если страницы нет в ОЗУ? (Страничный сбой)

Обратите внимание на строку 3 в нашей таблице. Что будет, если программа обратится к Странице 3, которая лежит на диске?

1. MMU смотрит в таблицу и видит статус: *"Этой страницы в ОЗУ нет, она на диске"*.
2. MMU не может сам пойти на диск (он слишком простой для этого). Он ставит процессор на паузу и генерирует аппаратное исключение — **Page Fault (Страничный сбой)**.
3. Операционная система просыпается, приостанавливает программу, лезет на SSD, находит нужную страницу и копирует её в любой свободный фрейм ОЗУ.
4. ОС обновляет Таблицу страниц (пишет туда новый номер фрейма).
5. Программа запускается с того же самого места и со второй попытки успешно читает данные. Для самой программы эта заминка незаметна (разве что компьютер на долю секунды «задумается»).

Плюсы и минусы страниц

- **Жирный Плюс:** Нет внешней фрагментации. Больше нет проблемы, когда «память есть, но она разбита на кусочки». Любая страница идеально поместится в любой свободный фрейм ОЗУ, потому что они одного размера.
- **Минус (Внутренняя фрагментация):** Если программе нужно выделить всего 1 байт памяти, ОС всё равно выделит ей целую страницу (4096 байт). Остаток в 4095 байт просто пропадет зря.

Если механика страниц, фреймов и таблицы страниц улеглась в голове, мы можем двигаться к **Шагу 3 — Сегментной организации**, чтобы понять, почему программистам одной страничной логики оказалось мало.

Или, если хотите, можем сначала решить короткую задачку «на бумаге», чтобы закрепить, как виртуальный адрес превращается в физический? Как вам лучше?

Отлично, двигаемся дальше! **Шаг 3 — Сегментная организация (Segmentation).**
Если страничная организация создавалась в угоду железу и математике (чтобы делить память на идеально равные кусочки), то сегментная организация родилась из потребностей **человека (программиста)** и логики работы самих программ.

В чем главная идея?

Программа никогда не состоит из однородной массы байт. С точки зрения программиста и компилятора, приложение — это набор **логических блоков** разного назначения и разного размера.
Сегментная организация отказывается от нарезки памяти «вслепую» по 4 КБ. Вместо этого она делит память процесса на **сегменты**, каждый из которых имеет свой смысл, свое имя и свой индивидуальный размер.
Типичная программа делится на следующие сегменты:

- 1. **Сегмент кода (Code / Text):** Здесь лежат сами инструкции процессора. Этот кусок памяти должен быть защищен — его можно только читать и выполнять, но ни в коем случае нельзя изменять.
- 2. **Сегмент данных (Data):** Здесь хранятся глобальные переменные программы. Его можно и читать, и менять.
- 3. **Сегмент стека (Stack):** Особая зона для временных переменных и адресов возврата из функций. Этот сегмент постоянно растет и сжимается в процессе работы.
- 4. **Сегмент библиотек:** Подключаемые внешние модули.

Как устроена адресация?

Поскольку сегменты имеют **разную длину**, виртуальный адрес в сегментной модели выглядит иначе. Он состоит из двух частей:

[**Номер сегмента** | **Смещение внутри сегмента**]

Вместо таблицы страниц операционная система создает **Таблицу сегментов**. В ней для каждого сегмента прописаны две важнейшие вещи:

- **Базовый адрес (Base):** Где именно в физической ОЗУ начинается этот сегмент.
- **Предел/Лимит (Limit):** Каков точный размер (длина) этого сегмента.

Пример таблицы сегментов для программы:

Номер сегмента	Описание	Базовый адрес в ОЗУ	Предел (Размер)	Права доступа
0	Код программы	10 000	1 500 байт	Только чтение / Выполнение
1	Глобальные данные	45 000	800 байт	Чтение / Запись
2	Стек	80 000	2 000 байт	Чтение / Запись

Как MMU проверяет и переводит адрес

Программа запрашивает данные: «Хочу прочитать байт со смещением 500 из сегмента 0 (Код)».

Блок MMU берет этот запрос и выполняет две проверки:

1. **Проверка на размер:** MMU смотрит в таблицу и проверяет: смещение 500 меньше, чем Предел сегмента 0(1500)? Да, всё в порядке. (Если бы программа попросила смещение 2000, MMU мгновенно заблокировал бы операцию и выдал знаменитую ошибку **Segmentation Fault** — признак того, что программа полезла за пределы своего сегмента).
2. **Проверка прав:** Программа хочет только *прочитать* данные? Да. В таблице написано «Только чтение». Проверка пройдена.

После этого MMU делает простую математику: берет Базовый адрес сегмента (10 000) и прибавляет к нему смещение (500). Физический адрес в ОЗУ — 10 500.

Почему сегментация — это круто? (Плюсы)

- **Жесткая безопасность и права доступа:** Можно аппаратно запретить изменять код. Если вирус попытается внедриться в сегмент кода и перезаписать его, процессор сразу его остановит.
- **Удобно делиться памятью:** Если вы запустите три копии одного и того же текстового редактора, операционной системе не нужно трижды загружать код программы в ОЗУ. Все три процесса могут использовать **один и тот же физический сегмент кода** (ведь он все равно не меняется), но у каждого будет свой личный сегмент данных и стека. Это экономит кучу оперативки.

Почему чистая сегментация проиграла? (Главный минус)

У сегментов есть один критический недостаток — **внешняя фрагментация памяти**. Так как сегменты имеют разный размер (один 500 КБ, другой 3 МБ, третий 12 МБ), со временем, когда программы запускаются и закрываются, физическая оперативная память становится похожа на швейцарский сыр. Она вся покрывается «дырами» случайных размеров. **Ситуация:** У вас есть 5 свободных «дыр» в памяти по 2 МБ каждая (всего 10 МБ свободной памяти). Вы запускаете программу, которой нужен один сплошной сегмент данных размером 5 МБ.

Итог: Программа не запустится! Свободная память в системе есть, но втиснуть в нее сплошной кусок на 5 МБ невозможно. Системе приходится тратить огромные ресурсы на «дефрагментацию» памяти (двигать чужие сегменты в ОЗУ, чтобы объединить свободные дыры).

Резюме

- **Страницы (Шаг 2):** Память делится на равные куски. Железу удобно, фрагментации памяти нет, но нет и логики.

- **Сегменты (Шаг 3):** Память делится на логические куски разного размера.

Программисту удобно, есть защита прав, но память жутко фрагментируется.

Логичный вопрос: как взять лучшее от обоих миров? Для этого инженеры придумали **Шаг 4 — Сегментно-страничную организацию**. Готовы посмотреть, как они скрестили эти две технологии?

Мы подошли к кульминации — **Шагу 4: Сегментно-страничной организации памяти**.

Именно по этому принципу устроена работа оперативной памяти во всех современных компьютерах, смартфонах и серверах (на архитектурах Intel, AMD и ARM). Это идеальный гибрид, который взял лучшее от обеих предыдущих технологий.

Главная идея: Матрешка

Инженеры поняли, что спорить между страницами и сегментами бессмысленно — нужно просто **вложить одно в другое**.

1. С точки зрения **программы и программиста**, память по-прежнему делится на крупные логические **сегменты** (сегмент кода, сегмент данных, стек). Это дает удобную защиту данных и изоляцию блоков.
2. Но с точки зрения **физической ОЗУ**, операционная система больше не пытается запихнуть эти сегменты на плату целиком. Вместо этого она берет каждый сегмент и внутри него нарезает память на стандартные одинаковые **страницы** по 4 КБ.

Физическая оперативка (ОЗУ) при этом по-прежнему состоит из стандартных **фреймов**.

Как устроен виртуальный адрес

Поскольку система стала двухступенчатой, виртуальный адрес теперь делится на три части:

[Номер сегмента | Номер страницы внутри сегмента | Смещение на странице]

Двойной перевод (Как работает MMU)

Когда процессору нужно прочитать данные по такому адресу, блок MMU делает два последовательных шага:

Шаг 1: Проверка сегмента

MMU идет в **Таблицу сегментов**.

- Она проверяет права доступа (можно ли читать/писать).
- Вместо «базового физического адреса» таблица сегментов теперь указывает на **Таблицу страниц для этого конкретного сегмента**. То есть у каждого сегмента появляется свой собственный каталог страниц.

Шаг 2: Поиск физической страницы

MMU берет из адреса *Номер страницы*, заглядывает в таблицу страниц этого сегмента и находит **Номер физического фрейма** в ОЗУ.

- К номеру фрейма прибавляется *Смещение*, и процессор наконец получает точный физический адрес нужного байта.

Почему это идеальное решение?

Этот гибрид полностью уничтожил главные недостатки обоих методов:

1. **Ф (Решена проблема сегментов):** Поскольку сегменты теперь состоят из страниц, их не нужно укладывать в ОЗУ одним сплошным куском. Страницы сегмента кода могут быть раскиданы по ОЗУ вперемешку со страницами сегмента стека. Память используется на все 100%, и никаких «дыр» в ОЗУ не образуется.
2. **Осталась железная защита (Плюс сегментов):** Мы все еще можем легко защитить весь код программы от изменения, просто выставив флаг «Только для чтения» на уровне сегмента.
3. **Файл подкачки стал мега-эффективным:** Если системе не хватает памяти, ей не нужно сбрасывать на диск весь огромный сегмент данных. Она может скинуть на SSD только одну конкретную страницу этого сегмента, к которой программа давно не обращалась.

Что в итоге?

Давайте соберем всю картину воедино, пройдя по нашей цепочке:

- **База:** Мы разделили адреса на виртуальные (для программ) и физические (для чипов).
- **Страницы:** Мы научились делить память на равные кубики по 4 КБ, чтобы эффективно заполнять ОЗУ.
- **Сегменты:** Мы научились группировать память по смыслу (код, данные), чтобы защитить программы от сбоев и вирусов.
- **Гибрид (Сегменты + Страницы):** Мы сгруппировали память по смыслу для программ, но внутри нарезали на кубики для железа.

Мы разобрали всю классическую теорию! Чтобы зафиналить тему виртуальной памяти, остался последний **Шаг 5: Реальность**. Там живут **многоуровневые таблицы** и супербыстрый кэш **TLB**, без которых современные 64-битные процессоры просто захлебнулись бы в расчетах.

Хотите заглянуть «под капот» современного процессора и узнать, как инженеры заставили эту сложную схему работать на гигантских скоростях?

Мы на финишной прямой — **Шаг 5: Реальность (Многоуровневые таблицы и TLB)**.

В теории сегментно-страничная модель выглядит идеально. Но когда инженеры попытались перенести её на современные 64-битные процессоры, они столкнулись с суровым математическим тупиком.

Давайте посмотрим, какие две проблемы возникли и как их гениально решили.

Проблема №1: Таблицы страниц стали слишком жирными

В старых 32-битных системах программа могла адресовать максимум 4 ГБ памяти. Если разбить 4 ГБ на страницы по 4 КБ, нам понадобится ровно 1 миллион строк в таблице страниц. Такая таблица весит около 4 МБ. Это терпимо.

Но современные процессоры — **64-битные**. Виртуальное адресное пространство одной программы стало астрономическим (даже с учетом ограничений архитектуры x86-64, реально используется 48 бит адреса — это 256 Терабайт).

Если бы мы создали одну плоскую таблицу страниц под такой объем, она бы весила **сотни Гигабайт**! То есть вся оперативная память компьютера уходила бы только на хранение самой таблицы страниц для одной-единственной программы.

Решение: Многоуровневые таблицы страниц (Дерево таблиц)

Инженеры решили не создавать всю таблицу сразу. Зачем хранить строки для адресов, которые программа даже не использует? Таблицу страниц превратили в многоуровневое дерево (в современных чипах Intel/AMD используется 4-уровневая или даже 5-уровневая структура).

- **Как это работает:** Вместо одной гигантской книги у нас есть маленькая «Главная таблица» (Каталог верхнего уровня). Она указывает на таблицы второго уровня, те — на третий, и только в самом конце находятся реальные номера фреймов.
- **В чем магия:** Если программа использует всего 10 МБ памяти, операционная система создает для неё только «Главную таблицу» и пару веточек. Все остальные терабайты неиспользуемого пространства вообще не занимают места в ОЗУ. Таблица растёт динамически, только когда программе это реально нужно.

Проблема №2: Катастрофическая потеря скорости

Плата за многоуровневые таблицы оказалась высокой. Представьте: процессору нужно прочитать всего 1 байт данных из ОЗУ.

Но чтобы узнать, где этот байт лежит, блоку MMU приходится:

1. Сходить в ОЗУ за таблицей 1-го уровня.
2. Сходить в ОЗУ за таблицей 2-го уровня.
3. Сходить в ОЗУ за таблицей 3-го уровня.
4. Сходить в ОЗУ за таблицей 4-го уровня.
5. И только после этого сходить в ОЗУ за самим байтом!

Вместо одного обращения к памяти мы получаем пять. Компьютер замедлился бы в 5 раз. Это катастрофа.

Решение: Аппаратный кэш TLB (Translation Lookaside Buffer)

Чтобы процессор не бегал по древовидным таблицам при каждом запросе, инженеры встроили прямо внутрь процессора (в блок MMU) сверхбыструю специализированную память — **TLB (Буфер перевода адресов)**.

- **Что это такое:** Это крошечный, но невероятно быстрый аппаратный кэш, который хранит историю последних переводов виртуальных страниц в физические фреймы.
- **Как это работает на практике:** Когда программа запрашивает адрес, MMU первым делом заглядывает в TLB.
 - В **95–99% случаев** нужный перевод адреса уже лежит там (это называется *TLB Hit* или попадание в кэш). Перевод происходит мгновенно, за доли такта процессора, без каких-либо походов в ОЗУ.

- Если программа обратилась к совершенно новому участку памяти, происходит *TLB Miss* (промах). Вот тогда MMU послушно идет блуждать по многоуровневым таблицам в ОЗУ, находит адрес, отдает его процессору и попутно **записывает этот маршрут в TLB**, чтобы в следующий раз выдать его мгновенно.

•

Итоговая архитектура памяти в вашем ПК

Теперь вы знаете всю цепочку изнутри. Когда вы запускаете, например, современную игру:

1. Программа видит красивую, непрерывную **виртуальную память**.
2. Код и данные игры разделены на **сегменты** (для безопасности и защиты прав доступа).
3. Эти сегменты разбиты на **страницы по 4 КБ**, чтобы железо могло раскидать их по ОЗУ как угодно, избегая фрагментации.
4. Адреса этих страниц хранятся в хитрых **многоуровневых таблицах**, чтобы экономить оперативку.
5. Процессор летает на огромной скорости, потому что блок MMU щелкает адреса через супербыстрый кэш **TLB**.
6. А если игре не хватит вашей физической ОЗУ, операционная система незаметно отправит редкие страницы отдыхать в **файл подкачки (Swap)** на SSD.

Мы полностью разобрали этот непростой путь от базовой концепции до устройства современных процессоров. Как вам такая картина? Если у вас остались вопросы по какому-то из шагов или вы хотите закрепить что-то конкретное, дайте знать.

Если вы изучаете это для учебы или работы, мы можем пройти по **типичным вопросам на экзаменах или собеседованиях** по этой теме. Будет ли это для вас актуально?